

PARTS LIST

Semiconductors:

- IC1 - P89V51RD2 microcontroller
- IC2 - L293D motor driver
- IC3 - MAX232 level converter
- IC4 - 7805, 5V regulator

Resistors (all $\frac{1}{4}$ -watt, $\pm 5\%$ carbon):

- R1 - 10-kilo-ohm

Capacitors:

- C1, C2 - 0.1 μ F ceramic disk
- C3 - 10 μ F, 16V electrolytic
- C4-C8 - 1 μ F, 16V electrolytic
- C9, C10 - 33pF ceramic disk

Miscellaneous:

- X_{TAL1} - 11.0592 MHz crystal oscillator
- M1, M2 - 9V, 100 rpm geared DC motor
- CON1 - 2-pin connector
- CON2 - DB9 connector
- S1 - Tactile switch
- 9V battery

Various Instructions to Move the Robot

Position of RED object	Instruction	Process
Red object up	a	Forward
Red object down	c	Backward
Red object right	b	Right
Red object left	d	Left
Red object at the centre	e	Stop

is connected to the robot through USB-to-serial converter. Controlling commands to the robot are sent via serial port and the signal levels are converted into 5V TTL/CMOS type by IC3. These signals are directly fed to microcontroller IC1 for controlling motors M1 and M2 to move the robot in all directions. Port pins P1.0 through P1.3 of IC1 are connected to the inputs of IN4 through IN1 of IC2, respectively, to give driving inputs. EN1 and EN2 are connected to Vcc to keep IC2 always enabled.

The application running on the computer interprets the position of the coloured object and sends corresponding commands to the robot through the serial port. As shown in the pictures in Fig. 1, if the operator stands in front of the computer's camera, holds a red-coloured object in his hand and raises his hand up, the MATLAB application running in the computer interprets the position of the coloured object and sends 'a' to the



Fig. 3: GUI of drive your robot



Fig. 4: Signal to stop the robot



Fig. 5: Signal to move forward

serial port. The robot is programmed to move forward if it receives 'a' from the serial port. Similarly, for other positions, the letters are listed in the table, which are sent through the serial port to the robot.

Software

The program for the robot is written in C language using Keil compiler and the hex file is programmed in the microcontroller chip with a suitable programmer. The application running on the computer to interpret the position of coloured object is designed using MATLAB. The application uses colour-detection algorithm to detect the coloured object and then finds out its coordinates to detect the position.

Colour detection algorithm. In colour-detection algorithm, the primary-colour (red, green and blue) objects can be detected easily. The algorithm used for colour detection actually works in separate steps as follows:

1. Take the snapshot (image) from the real video
2. Convert that original snapshot to grey

3. Extract the red coloured components from the original snapshot

4. Now subtract the red coloured components from the grey coloured snapshot

5. Remove the noise from the image by using filter command

6. Convert that filtered image into binary image and you will get an image that is bright at the place of red object

7. Then find the coordinates of the bright portion

MATLAB program. MATLAB provides a GUI development platform where engineers can develop an aesthetic graphic interface for their application. The program first initialises and starts the video by using the functions mentioned below:

```
vid = videoinput('winvideo',1,'YUY2_320x240');
set(vid, 'FramesPerTrigger', Inf);
set(vid, 'ReturnedColorspace', 'rgb')
vid.FrameGrabInterval = 2;
```

```
start(vid)
```

Some important functions used in the program are:

data = getsnapshot(vid). This function immediately returns single-image frame from the video input object

img = rgb2gray(data). Converts the true-colour image RGB to the greyscale intensity image

data(:, :, 1). Extracts all the red coloured components from the real image

Similarly, for green: **data(:, :, 2)** and for blue: **data(:, :, 3)**

diff_im = imsubtract(data(:, :, 1), rgb2gray(data)). Used to subtract red components from the grey image

B = medfilt2(diff_im, [3, 3]). Median filtering is a non-linear operation often used in image processing to reduce 'salt and pepper' noise. A median filter is more effective than convolution when the goal is to simultaneously reduce noise and preserve edges

diff_im = im2bw(diff_im, 0.18). Converts the grayscale image to a binary image. The output image 'diff_im' replaces all pixels in the input image with luminance greater than level with the value 1 (white) and replaces